

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- EXPERIMENTS

Course Code:	CSA1223	Course Title:	Computer Architecture
CO Assessed:	CO3	Assessment:	ASSESSMENT TOOL 1- EXPERIMENTS
Weightage:	40%	Total Marks:	100
CO3 Statement:	<i>Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT). (BL6)</i>		
Student Name:	B.Poorvaja	Reg.No.:	192473008
Department:	B.E-CSE(DS)	Date:	

ANNEXURE A

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.

REGISTER TRANSFER OPERATIONS WITH ARITHMETIC AND LOGIC OPERATIONS

AIM

To design and simulate a system that performs register transfer operations and basic ALU operations such as addition, subtraction, AND, and OR using multiple registers.

DESIGN

Consider four registers:

- **R1** – stores the first input
- **R2** – stores the second input
- **R3** – stores intermediate results
- **R4** – stores the final result

The ALU performs arithmetic and logical operations on the contents of the registers.

REGISTER TRANSFER OPERATIONS

Suppose:

- R1 = 15

- $R2 = 5$

The following operations can be performed:

Step	Operation	Description
1	$R3 \leftarrow R1$	Transfer contents of R1 to R3
2	$R3 \leftarrow R1 + R2$	Add R1 and R2
3	$R4 \leftarrow R1 - R2$	Subtract R2 from R1
4	$R3 \leftarrow R1 \text{ AND } R2$	Perform bitwise AND
5	$R4 \leftarrow R1 \text{ OR } R2$	Perform bitwise OR

For 8-bit registers:

$R1 = 00001111 = 15$

$R2 = 00000101 = 5$

ALU OPERATIONS

ADDITION:

$R3 = R1 + R2$

$= 15 + 5$

$= 20$

Binary:

00001111

+00000101

00010100

SUBTRACTION:

$R4 = R1 - R2$

$= 15 - 5$

$= 10$

AND:

00001111

AND 00000101

00000101 = 5

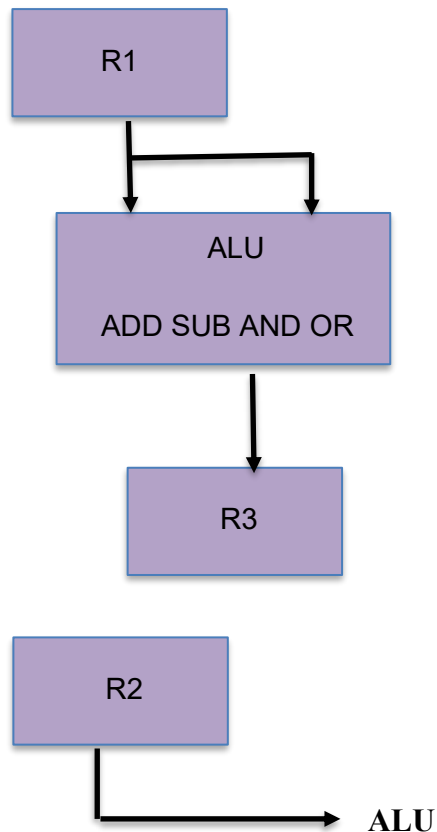
OR:

00001111

OR 00000101

00001111 = 15

BLOCK DIAGRAM



ANALYSIS

The registers provide temporary storage, while the ALU performs the required arithmetic and logical operations. Register-transfer operations allow data to move between registers under the control of the processor.

The experiment demonstrates that different operations can be selected by providing appropriate control signals to the ALU. This forms the basic datapath of a processor.

RESULT

The register transfer system successfully transfers data between registers and performs addition, subtraction, AND, and OR operations. The ALU acts as the main computational unit, while registers provide fast temporary storage.

2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyze how multiple bus structures improve parallel data transfer and reduce bottlenecks.

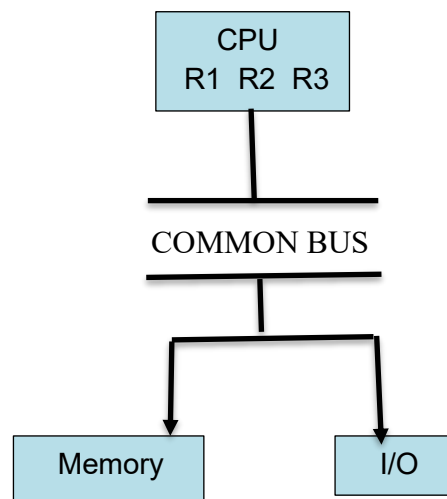
SINGLE-BUS AND MULTI-BUS ORGANIZATION

AIM

To model and compare single-bus and multi-bus computer organizations for transferring data between registers, memory, and I/O devices.

SINGLE-BUS ORGANIZATION

In a single-bus organization, all components communicate through one common bus.



For example:

$R1 \rightarrow \text{Bus} \rightarrow R2$

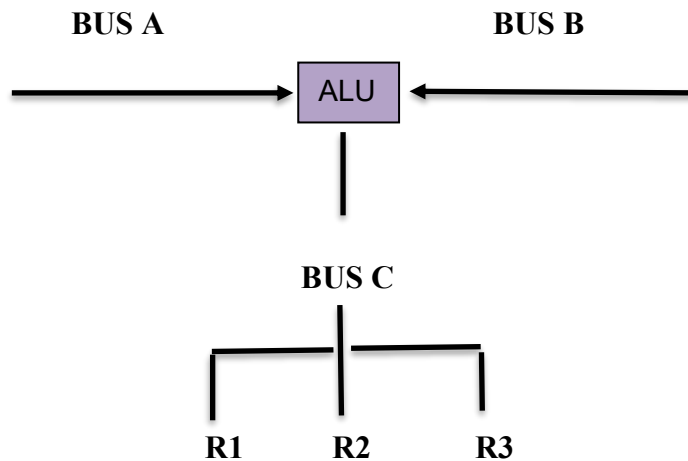
Only one major transfer can use the common bus at a time.

MULTI-BUS ORGANIZATION

A multi-bus system uses multiple buses simultaneously.

A three-bus organization can have:

- Bus A – first ALU input
- Bus B – second ALU input
- Bus C – ALU output



EXAMPLE

Suppose:

R1 = 20

R2 = 10

We need to perform:

$R3 \leftarrow R1 + R2$

SINGLE BUS

The operation requires multiple sequential transfers:

R1 → Bus

Bus → ALU

R2 → Bus

Bus → ALU

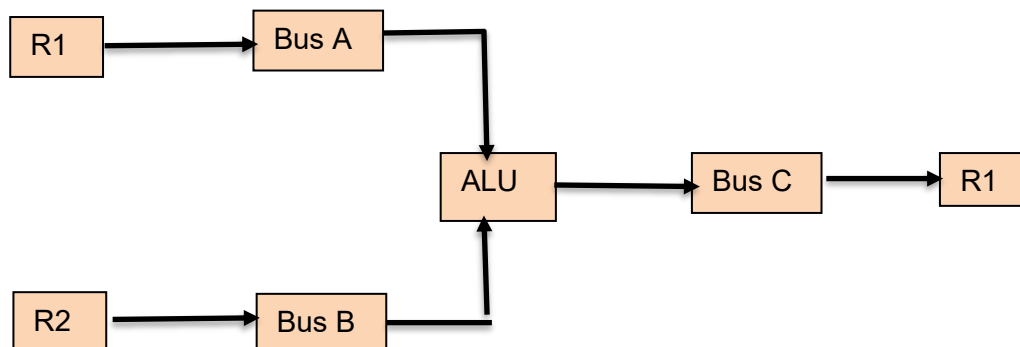
ALU → Bus

Bus → R3

Therefore, several clock cycles may be required.

MULTI-BUS

Two operands can be supplied simultaneously:



R1 and R2 can reach the ALU at the same time.

COMPARISON

Feature	Single Bus	Multi Bus
Number of buses	One	Two or more
Data transfer	Mostly sequential	More parallel
Hardware complexity	Low	Higher
Cost	Lower	Higher
Speed	Lower	Higher
Bottleneck	Common bus	Reduced
Parallel operations	Limited	High

ANALYSIS

The single-bus architecture is simpler and cheaper, but the common bus becomes a bottleneck when several components require data transfer.

The multi-bus architecture allows simultaneous data transfers, reducing the number of clock cycles required for an operation. However, it requires additional hardware and control logic.

RESULT

The multi-bus organization provides better performance because multiple operands and results can be transferred simultaneously. Therefore, it is more suitable for high-performance processors, while single-bus systems are simpler and economical.

3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.

FIVE-STAGE INSTRUCTION PIPELINE

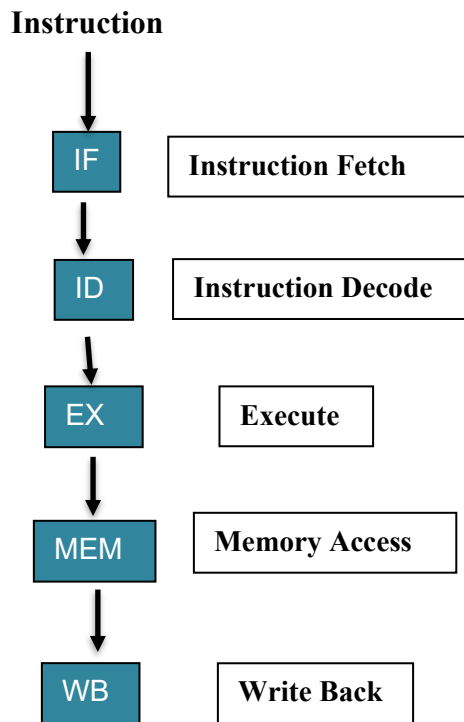
AIM

To simulate a five-stage instruction pipeline and compare its performance with a non-pipelined processor.

The five stages are:

1. IF – Instruction Fetch
2. ID – Instruction Decode
3. EX – Execute
4. MEM – Memory Access
5. WB – Write Back

PIPELINE STRUCTURE



Consider five independent instructions:

I1: ADD R1,R2,R3

I2: SUB R4,R5,R6

I3: AND R7,R8,R9

I4: OR R10,R11,R12

I5: ADD R13,R14,R15

PIPELINE EXECUTION

Clock Cycle	I1	I2	I3	I4	I5
1	IF				
2	ID	IF			
3	EX	ID	IF		
4	MEM	EX	ID	IF	
5	WB	MEM	EX	ID	IF
6		WB	MEM	EX	ID
7			WB	MEM	EX
8				WB	MEM
9					WB

Thus, 5 instructions are completed in 9 cycles.

NON-PIPELINED EXECUTION

In a non-pipelined processor, each instruction completes all five stages before the next instruction begins.

For five instructions:

$$5 \times 5 = 25 \text{ cycles}$$

SPEEDUP

Pipeline execution:

9 cycles

Non-pipeline execution:

25 cycles

Therefore:

$$\text{Speedup} = 25/9$$

$$\text{Speedup} \approx 2.78$$

So, the pipelined system provides approximately **2.78× speedup** for these five instructions.

THROUGHPUT

Non-pipelined processor:

5 instructions / 25 cycles

$$= 0.2 \text{ instructions/cycle}$$

Pipelined processor:

5 instructions / 9 cycles

$$\approx 0.556 \text{ instructions/cycle}$$

Therefore, pipeline throughput is significantly higher.

ANALYSIS

Pipelining improves processor performance by overlapping different stages of multiple instructions. While one instruction is being executed, another can be decoded and another can be fetched.

The main advantage is increased instruction throughput rather than reducing the execution time of an individual instruction.

RESULT

The five-stage pipeline successfully executes multiple instructions concurrently. Compared with the non-pipelined system, the pipelined processor provides higher throughput and approximately 2.78× speedup for the given instruction sequence.

4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.

PIPELINE HAZARDS AND THEIR SOLUTIONS

AIM

To create a pipeline scenario containing data, control, and structural hazards, analyze their effects, and apply stalling, forwarding, and branch prediction techniques.

The question specifically requires these three hazard types and their solutions.

1. DATA HAZARD

Consider:

I1: ADD R1, R2, R3

I2: SUB R4, R1, R5

Here, I2 requires the value of R1, which is produced by I1.

This creates a Read After Write (RAW) hazard.

Without Forwarding

I2 must wait until I1 writes R1.

Cycle: 1 2 3 4 5 6

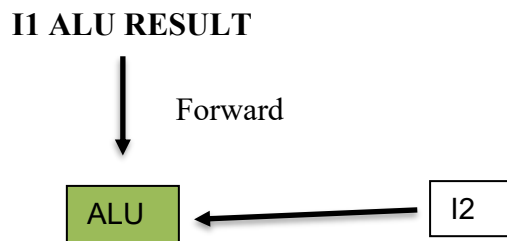
I1: IF ID EX MEM WB

I2: IF ID ST ST EX

The pipeline stalls, reducing performance.

With Data Forwarding

The result from I1's EX/MEM pipeline register can be directly forwarded to the ALU input of I2.



This avoids unnecessary waiting.

2. CONTROL HAZARD

Consider:

I1: BEQ R1,R2,LABEL

I2: ADD R3,R4,R5

I3: SUB R6,R7,R8

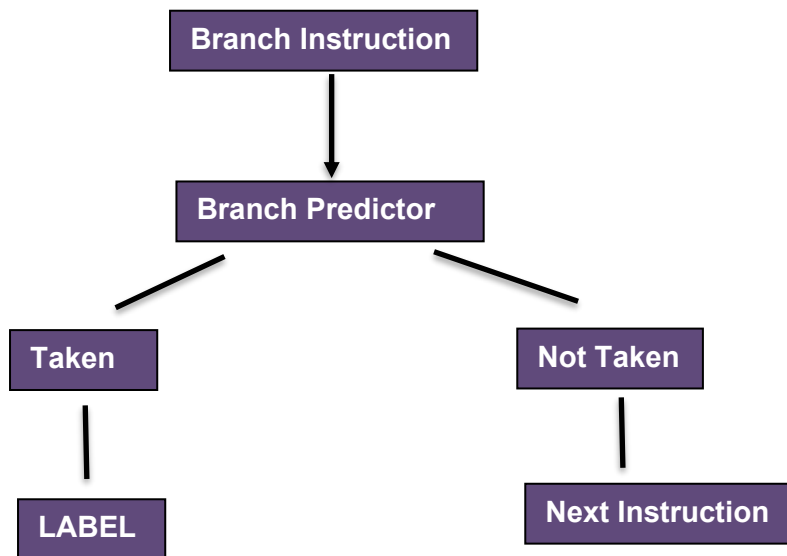
The processor does not immediately know whether the branch will be taken.

If the branch is taken, instructions fetched after the branch may be incorrect.

Solution: Branch Prediction

The processor predicts whether the branch will be taken.

Example:



If the prediction is correct, pipeline execution continues without major delay.

If the prediction is wrong, incorrectly fetched instructions are removed from the pipeline, causing a branch misprediction penalty.

3. STRUCTURAL HAZARD

A structural hazard occurs when two pipeline stages require the same hardware resource at the same time.

For example:

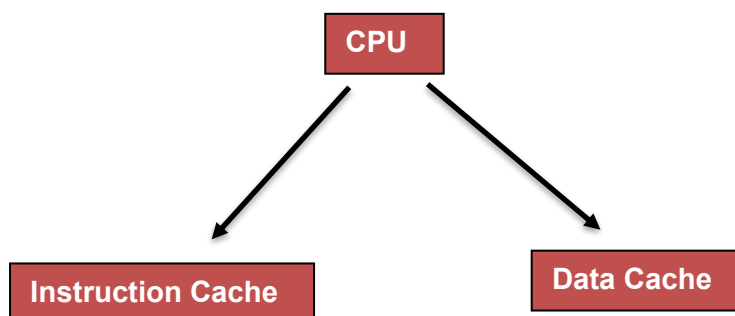
Instruction 1 → Memory access

Instruction 2 → Instruction fetch

If the processor has only one memory unit, both operations compete for the same resource.

Solution

Use separate instruction and data memories/caches:



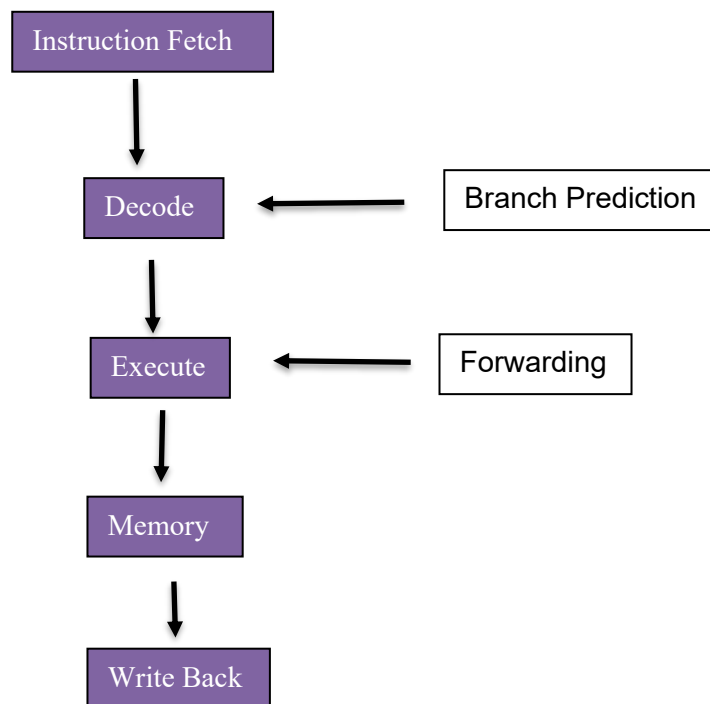
Now instruction fetching and data access can happen simultaneously.

Another solution is to stall one instruction until the resource becomes available.

HAZARD COMPARISON

Hazard	Cause	Example	Solution
Data	Dependency between instructions	RAW	Forwarding/Stalling
Control	Branch/jump	BEQ	Branch prediction
Structural	Resource conflict	Same memory	Duplicate resources/Stalling

OVERALL PIPELINE



ANALYSIS

Pipeline hazards prevent the processor from achieving ideal one-instruction-per-cycle execution. Forwarding reduces data stalls, branch prediction reduces control-hazard penalties, and additional hardware resources reduce structural conflicts.

A combination of these techniques allows the processor to maintain high pipeline utilization.

RESULT

The three major pipeline hazards were successfully identified and appropriate solutions were applied:

- Data hazard → Forwarding/Stalling
- Control hazard → Branch Prediction
- Structural hazard → Additional resources/Stalling

These techniques improve pipeline efficiency and reduce execution delays.

5. Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

SUPERSCALAR, OUT-OF-ORDER, SPECULATIVE EXECUTION AND SMT

AIM

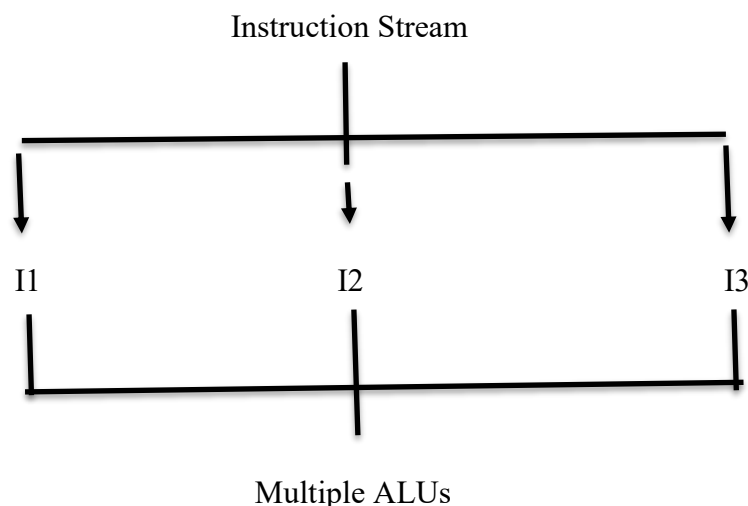
To analyze a processor that supports superscalar execution, out-of-order execution, speculative execution, hyper-threading and simultaneous multithreading (SMT) and evaluate their effect on instruction throughput and resource utilization.

The assessment specifically asks for comparison of these techniques and their effect on parallel instruction execution.

1. SUPERSCALAR EXECUTION

A superscalar processor can issue multiple instructions in one clock cycle.

For example, a 4-wide processor can potentially issue four independent instructions per cycle.



EXAMPLE:

I1: ADD R1,R2,R3

I2: SUB R4,R5,R6

I3: AND R7,R8,R9

I4: OR R10,R11,R12

Since these instructions are independent, multiple instructions can execute simultaneously.

2. OUT-OF-ORDER EXECUTION

In a conventional processor, instructions normally execute in program order.

In an out-of-order processor, an instruction that is ready can execute before an earlier instruction that is waiting for data.

EXAMPLE:

I1: LOAD R1, Memory

I2: ADD R2, R3, R4

I3: SUB R5, R6, R7

If I1 is waiting for memory, I2 and I3 can execute first if their operands are available.

Program order:

I1 → I2 → I3

Actual execution:

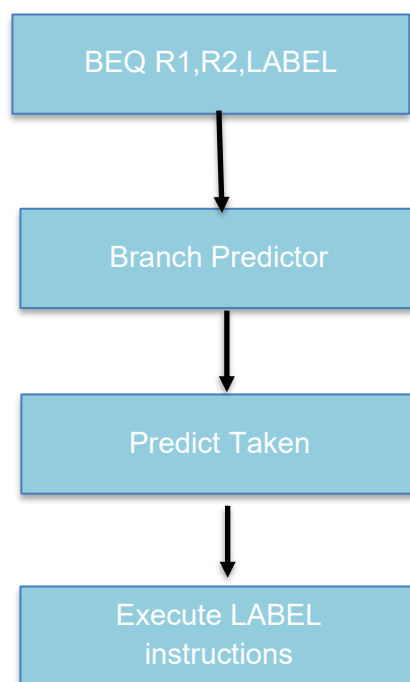
I2 → I3 → I1

This keeps execution units busy.

3. SPECULATIVE EXECUTION

Speculative execution allows the processor to execute instructions before it knows with certainty that they are required.

For example:



If the prediction is correct:
Continue execution

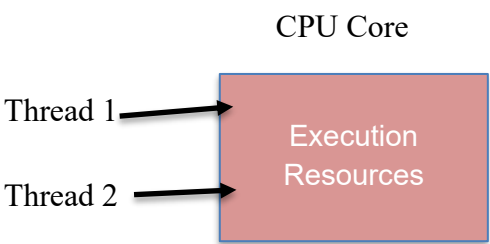
If the prediction is wrong:
Discard speculative results
Fetch correct instructions

Speculative execution therefore improves performance when branch predictions are accurate.

4. HYPER-THREADING / SMT

Simultaneous Multithreading (SMT) allows multiple instruction streams or threads to share processor execution resources.

A simplified model is:



When Thread 1 is waiting for memory, resources that would otherwise remain unused can execute instructions from Thread 2.
This improves resource utilization.

COMPARISON

Technique	Main Idea	Main Benefit
Superscalar	Execute multiple instructions per cycle	Higher instruction through
Out-of-order	Execute ready instructions before waitin	Reduces idle execution u
Speculative executi	Execute predicted instructions	Reduces branch delays
Hyper-threading	Multiple logical threads on one core	Better utilization of resou
SMT	Multiple instruction streams concurrently	Higher overall throughpu

EXAMPLE OF COMBINED OPERATION

Consider:

I1: LOAD R1, [Memory]

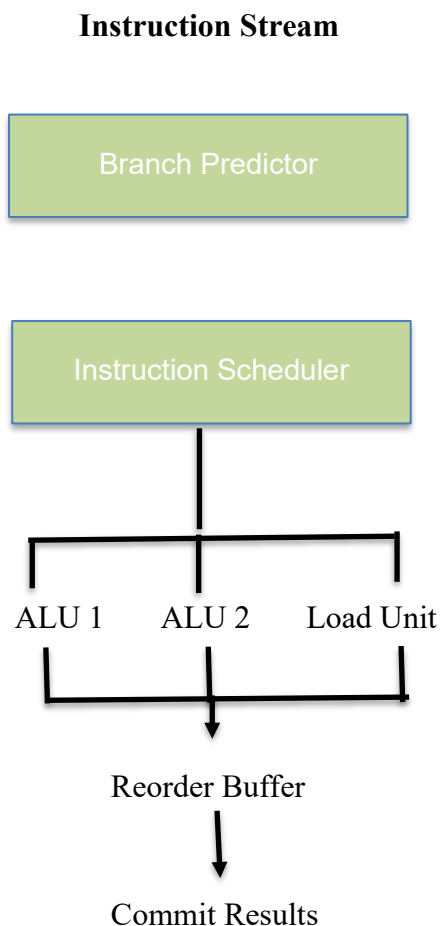
I2: ADD R2,R3,R4

I3: BEQ R5,R6,L1

I4: SUB R7,R8,R9

I5: AND R10,R11,R12

A modern processor can operate as follows:



- **Superscalar execution** allows multiple instructions to be issued.
- **Out-of-order execution** allows ready instructions to execute first.
- **Speculative execution** allows predicted branch instructions to execute.
- **SMT** allows instructions from multiple threads to use available resources.

PERFORMANCE ANALYSIS

Assume a processor executes 100 instructions.

A single-issue processor might ideally require approximately:

100 cycles

A 4-wide superscalar processor can theoretically process up to:

4 instructions/cycle

Therefore, the ideal minimum is approximately:

$$100/4 = 25 \text{ cycles}$$

In practice, the actual number will be higher because of:

- Data dependencies
- Cache misses
- Branch mispredictions
- Pipeline stalls
- Limited execution resources

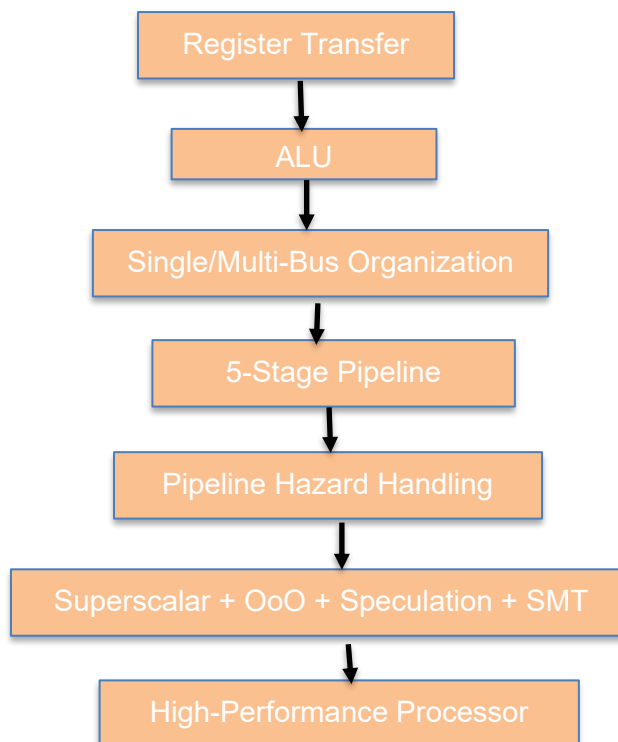
Thus, superscalar and out-of-order execution improve throughput, while speculative execution reduces the effect of control dependencies. SMT can further improve utilization when one thread cannot fully occupy the processor.

RESULT

The analysis demonstrates that modern processors combine superscalar execution, out-of-order execution, speculative execution, and SMT to execute multiple instructions concurrently. These techniques increase instruction throughput and improve utilization of processor resources, resulting in significantly better performance than a simple sequential processor.

CONCLUSION

The five experiments demonstrate the progression from a basic processor datapath to modern high-performance processor organization:



These experiments collectively demonstrate how datapath organization, pipelining, hazard resolution, parallel instruction execution, and multithreading are used to improve processor performance and resource utilization.

Code of Conduct:

I __B.Poorvaja(192473008)__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Understanding of Concepts (20%)	Experimental Design (20%)	Use of Tools (25%)	Analysis of Results (20%)	Documentation (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies(5)	Good understanding with proper integration of most concepts(4)	Basic understanding; limited or partial integration(3)	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution(6)	Correct implementation with minor issues(4)	Basic implementation with errors or inefficiencies(3)	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration(4)	Appropriate use of tools with correct execution(3)	Limited or inconsistent use of tools(2)	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results(3)	Good analysis with reasonable interpretation(2)	Basic analysis with limited insights(1)	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results(2)	Organized documentation with necessary details(1.5)	Basic documentation with missing details(1)	Poor or incomplete documentation